

PHILIPS Biosensing Platform

DG1901 PSP Library Integration Guide

Rev. 1.3 — 14-Jul-2023

Approved

Philips Confidential

Document information

Info	Content
Keywords	Philips Biosensing Platform, PSP, PPG, Library
Abstract	Integration Guideline for the PSP Library

PHILIPS

Revision history

Rev	Date	Author	Description
1.0	20-Oct-2021	JaWo	Initial version
1.1	25-Nov-2021	JaWo	Add green and ambient required for SpO2 measurement
	24-May-2022	JaWo	Update MIPS for different packages
	07-Nov-2022	JaWo	Correct typos Update Disclaimer Update Disclaimer, update date format
1.2	13-Feb-2023	JaWo	Correct typo for barometers range Update ACC range for Fall Occurrence Update MCU resources requirement Add remark for optional input metrics not in ListRequiredMetrics Update wiff format description
1.3	14-Jul-2023	JaWo	Update platform name, document reference version number Remove Heart Rhythm Type Update memory foot print Add CSV file format for offline processing

CoreKinect LLC

Table of Contents

1	Introduction	4
1.1	Purpose.....	4
1.2	Scope.....	4
1.3	References.....	4
1.4	Software Reference	5
1.5	Definitions, Acronyms and Abbreviations.....	5
2	System Overview and Requirements	6
2.1	Sensor Signal Requirements.....	10
2.1.1	Requirements of PPG Signal to PSP Library	10
2.1.2	Requirements of 3D Accelerometer Signal to PSP Library	11
2.1.3	Requirements of Skin Conductance Signal to PSP Library	12
2.1.5	Requirements of Gyroscope Signal to PSP Library.....	13
2.1.6	Requirements of Barometer Signal to PSP Library	13
2.2	Host Microcontroller Requirements	13
2.3	PSP library Overview	14
2.4	Memory Requirements	15
2.5	Sensor Signal Requirements for Metrics	16
3	Generic Guidelines for Integrating PSP library	19
3.1	Software Integration Flow.....	19
3.2	Operation Flow of PSP library	20
3.2.1	Life Cycle Control - Fresh start.....	20
3.2.2	Life Cycle Control - Restart maintaining history	21
3.2.3	Calling Protocol Description	23
3.3	PSP API Function Usage Example	24
3.3.1	PSP_GetVersionInfo().....	24
3.3.2	Library Instance Parameters	24
3.3.3	PSP_GetDefaultParams().....	25
3.3.4	PSP_Initialise().....	26
3.3.5	PSP_EnableMetrics()	26
3.3.6	PSP_ListRequiredMetrics().....	27
3.3.7	PSP_SetMetric()	27
3.3.8	PSP_Process().....	29
3.3.9	PSP_ListUpdatedMetrics().....	30
3.3.10	PSP_GetMetric()	31
3.3.11	PSP_DisableMetrics()	31
3.3.12	PSP_Terminate()	32
3.4	Sensor Driver	32
3.4.1	Driver for PPG front End	32
3.4.2	Driver for Accelerometer	33
3.4.2.1	ACC Sensor Orientation Adjustment.....	33
3.5	Data Logging.....	34
3.5.1	Legacy WIFF binary format Overview	36
3.6	Library Deployment	37
3.6.1	Loading Binary library image.....	37
3.6.1.1	In Keil uVision5 environment.....	38
3.6.1.2	In IAR Embedded Workbench environment	39
3.6.2	Including header and helper files	43
A	Appendix: Typical Integration Issues.....	44
A.1	Linking Library in Keil	44

1 Introduction

Philips Biosensing Platform library offers robust and accurate healthcare solutions in wrist-worn devices. The solution is supported by Philips' deep experience in advanced healthcare biometrics technology.

PSP library provides a wide variety of metrics as specified in [SY1902]. System integrators can license specific metrics from Philips and integrate into their products.

1.1 Purpose

The purpose of this document is to describe how system integrators can integrate the PSP library to their target platforms, to assist them to build the system from the software point of view.

1.2 Scope

The scope of this document relates to PSP library integration. It applies to PSP 6.1.9. Newer versions of the library might not be fully covered by this version of the document. Certain peripheral functionalities, like LED power control, Analog Front End driver and accelerometer driver, are to be implemented by the system integrators and will not be fully covered in this document. However, where such components impact the usage of the PSP library, the required interactions with the library are explained along with examples of usage where applicable.

In this document, both hardware and software aspects are discussed. Additional mechanical and electrical information is covered in the electromechanical guide. The minimum hardware and system requirements to run the library are presented and the implications on the available functionality and/or performance are explained. Knowledge of software design (preferably embedded) and software library integration is fundamental to understand this document. Knowledge of hardware design is not strictly necessary, but it might be helpful to understand certain aspects.

1.3 References

Document ID	Title	Version
SY1901	PSP API Specification	1.3
SY1902	PSP Metrics Specification	1.3
SY1903	PSP-S API Specification	1.2
DG1903	PSP Biosensing by PPG Integration Guide	1.2
PRE13-13-583	PRE13-13-583_C07S035_WeST_Intercgangeable_File_Format	2.1

1.4 Software Reference

File	Contents
psp.h	PSP functions & definitions
psp_map.h	PSP functions physical memory addresses
psp.c	PSP function implementations, connecting the prototypes in psp.h to the addresses in psp_map.h
fx_datatypes.h	datatype definitions
psp.bin	executable binary code (binary file format)
psp.hex	executable binary code (Intel Hex file format)

1.5 Definitions, Acronyms and Abbreviations

Abbreviation	Description
ACC	Acceleration signal obtained by an accelerometer
AD	Analog to Digital
ADC	Analog to Digital Converter
AGB	Accelerometer, Gyroscope and Barometer
API	Application Programming Interface
App	An application, especially as downloaded by a user to a mobile device.
Bluetooth	A standard for the short-range wireless interconnection of mobile phones, computers, and other electronic devices.
BT	Bluetooth
CPU	Central Processing Unit
CSV	Comma Separated Format
DAC	Digital-to-Analog converter
EDA	Electrodermal Activities
Footprint	Refers to the amount of used memory and/or processing power.
FT format	Philips internal data text format
GSR	Galvanic Skin Response
Gyro	Gyroscope
HR	Heart Rate
IRQ	Interrupt Request
MCU	Micro-controller
MIPS	Million Instructions per Second. Unit used to represent CPU load.
PPG	Photoplethysmogram, the signal obtained by an optical heart-rate sensor.
PSP	Philips Biosensing Platform
PSP-S	Philips Biosensing Platform Sleep
RAM	Random Access Memory
RHR	Resting Heart Rate
SC	Skin Conductance

2 System Overview and Requirements

PSP is a collection of product lines. In PSP 6.1, it includes:

- Biosensing by PPG
- Sleep Sensing
- Biosensing by EDA
- Fall Detection by AGB

The metrics included in each product line are listed below.

Product Lines	Output Metrics / Features
Biosensing by PPG	Heart Rate
	Low Power Heart Rate
	Resting Heart Rate
	Heart Beat Time Stamp
	Private Data (for Sleep Stages)
	Respiration Rate
	Active Energy Expenditure
	Low Power Active Energy Expenditure
	Activity Type
	Cadence
	Speed
	VO2max
	Fitness Index
	SpO2
	Skin Proximity
	Activity Count
	Stress Level Heart Rate
Sleep Sensing	Sleep Stages
Biosensing by EDA	Stress Level Skin Conductance
	Cognitive Zone
	Skin Proximity
Fall Detection by AGB	Fall Occurrence

During the evaluation phase, five evaluation library packages are pre-defined for system integrators' evaluation. They are described in the following table.

Output Metrics / Features	Evaluation Package 1	Evaluation Package 2	Evaluation Package 3	Evaluation Package 4	Evaluation Package 5
Heart Rate	✓	✓			
Low Power Heart Rate	✓	✓			
Resting Heart Rate	✓	✓			
Heart Beat Time Stamp	✓	✓			
Sleep Stages					✓
Private Data (for Sleep Stages)	✓	✓			
Respiration Rate	✓	✓			
Active Energy Expenditure	✓	✓			
Low Power Active Energy Expenditure	✓	✓			
Activity Type	✓	✓			
Cadence	✓	✓			
Speed	✓	✓			
VO2max	✓	✓			
Fitness Index	✓	✓			
SpO2	✓	✓			
Skin Proximity	✓	✓	✓		
Activity Count	✓	✓			
Stress Level Heart Rate		✓			
Stress Level Skin Conductance	✓		✓		
Cognitive Zone	✓		✓		
Fall Occurrence				✓	

After the evaluation phase and system integrators are satisfied with the library performance, an official production version library will be delivered to the system integrators, in which the library includes only the licensed metrics as agreed in the Software Licensing Agreement signed between Philips and the licensee (can be system integrator or brand owner). Then this library can be used in the final products. Notice that the evaluation library is only meant to be used for evaluation only.

Biosensing by PPG uses an optical PPG sensor to accurately track blood volume variations and an accelerometer to track body motion. A proprietary algorithm extracts and processes the sensor data, factoring out body motion, pigmentation, and ambient lighting to produce highly accurate results.

Biosensing by EDA uses the electrodermal activities sensor to measure the skin conductance to predict the stress level and cognitive zone of a person. This metric is a collection of output including the Cognitive Zone, Predictive Cognitive Zone, Predictive Cognitive Zone Transition Time, Cognitive Zone in 1 hour and Cortisol Contribution. Note that the unit for Cortisol Contribution is arbitrary. The technology is based on historic data of the person and algorithm adapts to a specific person over some days.

The library comes in certain deployment formats, suited for specific computational platforms. The actual ways to call the API of the library on different target platforms may be slightly different. One of the default deployment formats is “C Library Image for ARM Cortex M4[F]”.

The captioned deployment format of the library suits for embedded systems with a micro-controller (MCU) ARM-M4F. The library is provided in binary form and the calling functions of APIs are provided in C compliant code for easy integration into various C compatible platforms and common tools chain for MCU, like IAR and Keil.

Philips provides the PSP library which relies on the various sensors signal such as PPG signal, Skin conductance signal and acceleration (ACC) data provided by peripheral sensors. The system integrators should build the MCU with driver to control the required sensors in order to capture the sensor signals and convert to required formats for the library. The MCU of the system is also responsible to feed required sensors signals to the library and control the library to process the inputs and produce result of enabled metrics.

The configuration of a wearable device with the PSP library is depicted in Fig 1. The system consists of three parts: 3rd party sensors, host MCU and mobile device (optional). The sensors include PPG front-end sensor, EDA front-end sensor, 3-axis Accelerometer etc depending on the metrics required. The host MCU includes hardware interfaces to connect to the sensors, drivers to interface the sensors and application program to control the program flow of the PSP library. PSP Sleep Sensing library is a necessity to enable Sleep Stages metric while mobile device is a possible host for PSP-S library. For example, PSP-S library can be alternatively hosted on the Android OS of wearable device if there is sufficient computing resource. Refer to [SY1903] for more details.

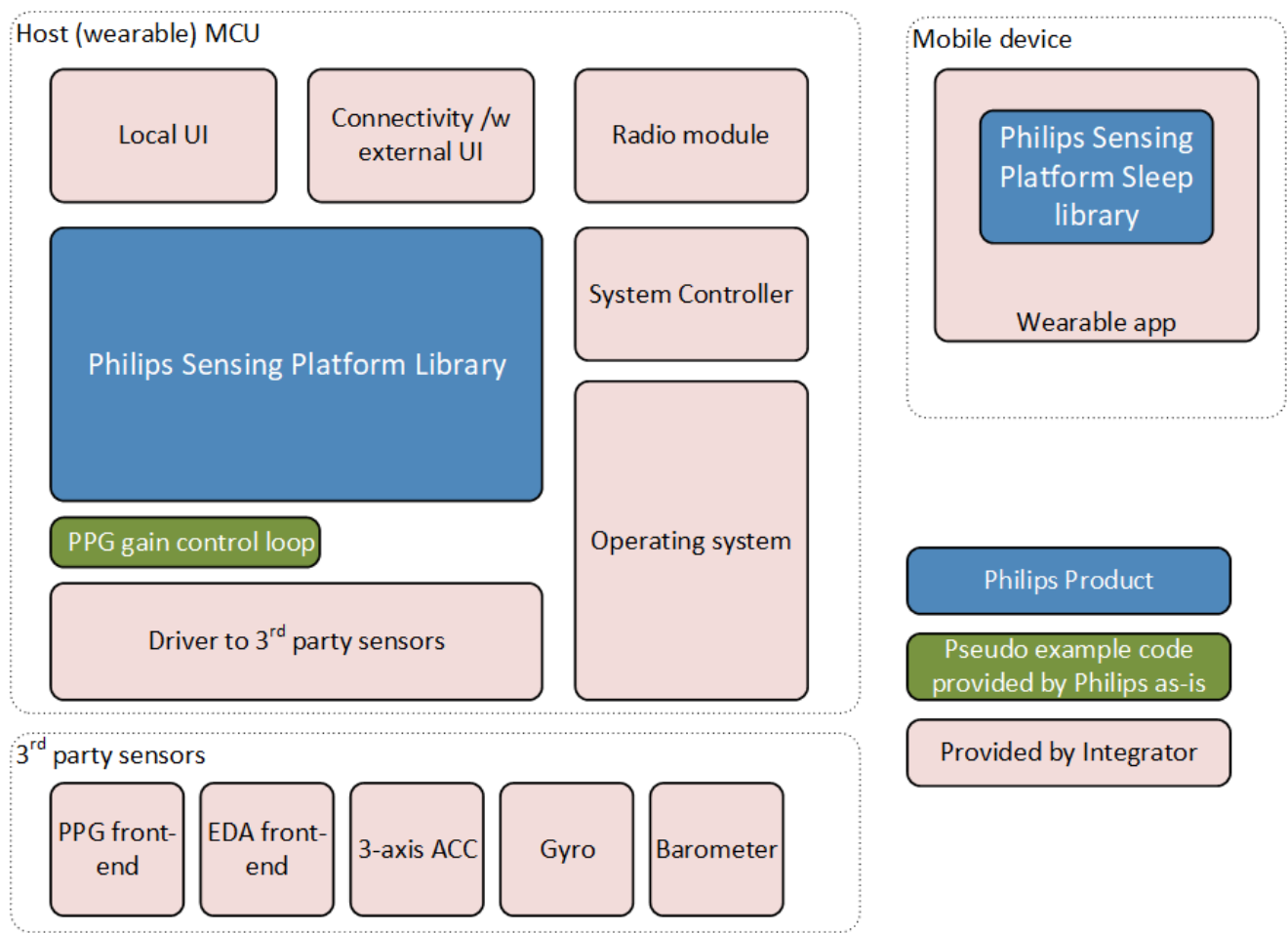


Fig 1. The configuration of a wearable device with PSP Library

Philips is offering the PSP library as an official product. The example codes (if any) such as the PPG gain control loop are provided as-is and no guarantee of robustness or product quality. System integrators can have the flexibility to further improve the example codes to fit for own requirements. The system integrators have to build the required functions like interfaces for the PPG front-end and accelerometer, the controls for the system and UI interfaces. PSP-S library is for Sleep Stages analysis and is a separate library under the PSP from Philips running in mobile device. For the rest of the components in the system, they are implemented and owned by the customer or system integrator.

To summarize, the software function and responsibility of the system components are described in Table 1.

Component	Description
PSP library	Provided by Philips, performing the core metrics calculation
Sensor Drivers	Built by system integrators to capture sensor data, convert to required formats and signal range
System Controller in MCU	Built by system integrators on the MCU to feed sensor data to the library and control the library to product results, maintain the life cycle of the PSP library

Table 1 : Software function and responsibility of system components

2.1 Sensor Signal Requirements

2.1.1 Requirements of PPG Signal to PSP Library

When interfacing the PPG front-end, the Analog front-end (AFE) provides DAC control of LED power and ADC output of analog PPG signal picked up from photodiode. The AFE provides the PPG signals with ambient light canceled and sends the raw data to main MCU typically through SPI or I2C serial interface. However, the PPG signal may not match with the input requirements for the PSP library and may be necessary to convert before feeding to the library. The ambient channel is required and it will improve the sensing of Skin Proximity (on-skin and off-skin). If Ambient signal is not available in the front end, set the value to zero but this metric is still required to feed to the PSP library. The PPG input requirements to the library are described in Table 2.

Parameter	Requirement
Sampling Rate	32 Hz
Dynamic Range	0 .. ($2^{16}-1$)
Resolution	16 bits unsigned (0 to 65535)
Signal to Noise Ratio	Green channel >80dB Red and Infra channel >85dB
PPG Signal	Ambient cancelled
Ambient Signal	Optional
LED power / PD gain control	Control interface available on chipset or automatically control by chipset

Table 2 : Library Input Requirements for PPG Signal

The Analog front-end should also contain configurable LED drivers in which the power output is controlled by the host MCU. With a controllable photodiode gain, it could form an adaptive control loop for different skin type persons under different use cases. Note that the adaptive control may only take place in certain moments in time. Too much change in the control loop may disrupt the continuity of the PPG signal. Alternatively, the Analog front-end may also include automatic control to adjust the LED/PD setting to adjust for different skin type conditions.

2.1.2 Requirements of 3D Accelerometer Signal to PSP Library

The 3D Accelerometer is designed to measure the motion of end users. It reports the proper acceleration over 3 axis. The ACC signals enable the motion related metrics as well as the motion artifacts compensation for some metrics such as heart rate. The 3D Accelerometer communicates with the host MCU typically with SPI or I2C serial interface. It is recommended to make use of the on-chip FIFO to accumulate the ACC samples for every second if possible. The ACC data transmission from accelerometer to host MCU should be preferably once per second in order to minimize the time and power consumption. In case of the FIFO depth is not available to accumulate the ACC samples for one second, the ACC data transmission may occur in few times per second and prevents any FIFO overflow.

The ACC signal (except for Fall Occurrence) to the PSP library should meet the requirements described in Table 3.

Parameter	Requirement
Sampling Rate	32 Hz
Dynamic Range	+/-8 g
Resolution	13 bits signed, 1/512 g/unit (-4096 to +4095)
Sensor Noise	<6mg RMS
Allowed 0g Offset	<150mg on any axis

Table 3 : Library Input Requirements for ACC signal (Except Fall Occurrence)

13 bits signed resolution is required for the ACC signals, if the use case is calculating the Cadence for cycling, integrator can choose to use an accelerometer such as BMA280 which provides better noise performance. In case the resolution and sample rate do not match signed 13bit at 32Hz, the ACC signal should be converted to signed 13bit at 32Hz. Care should be taken in the conversion to prevent aliasing effect.

The maximum allowed zero offset of the accelerometer is +/-150 mg. The dynamic range will be lower than +/-8 g if the allowed zero offset is exceeded. If the accelerometer offset is larger than 150mg, system integrators are required to perform offset calibration for the accelerometer. Accelerometer supplier typically provides the method for offset calibration.

The ACC signal for Fall Occurrence to the PSP library should meet the requirements described in Table 4.

Parameter	Requirement
Sampling Rate	50 Hz
Dynamic Range	+/-16g
Resolution	15 bits signed, 1/1024 g/unit (-16384 to +16383)
Sensor Noise	<6mg RMS
Allowed 0g Offset	<150mg on any axis

Table 4 : Library Input Requirements for ACC signal used in Fall Occurrence

In PSP 6.1, Fall Occurrence cannot be combined with other metrics in the same PSP package, so system integrators should only need to send the ACC signal (Acceleration format ID 0x70) as specified in [SY1902] when using Fall Occurrence. Otherwise, send the ACC signal (Acceleration format ID 0x6E) as specified in [SY1902] to PSP library. Note that if the accelerometer in reality exceeds +/-16g, the signal will clip and will degrade.

2.1.3 Requirements of Skin Conductance Signal to PSP Library

The skin conductance should be feeding to the PSP library according to the following. Note that usually the skin conductance can be measured by the GSR sensor. Conversion is required to convert the ADC value to skin conductance value.

The skin conductance signal to the PSP library should meet the requirement as in Table 5.

Parameter	Requirement
Sampling Rate	25 Hz
Dynamic Range	0 to 1677 μ S (Siemens)
Resolution	24 bits unsigned, 0.1nS/unit (0 to 16,777,216)
Signal to Noise Ratio	>100 dB
Sensor Noise and Interference	< 1 nS pkpk
Linearity	<0.003

Table 5 : Library Input Requirements for Skin Conductance

2.1.5 Requirements of Gyroscope Signal to PSP Library

The gyroscope signal to the PSP library should meet the requirement as in Table 6.

Parameter	Requirement
Sampling Rate	50 Hz
Dynamic Range	+/-2000 dps
Resolution	16 bits signed, 0.061 dps/unit (-32768 to +32767)
Sensor Noise	< 0.1125 dps RMS
Angular Rate 0 offset	< 0.75 dps

Table 6 : Library Input Requirements for Gyroscope

2.1.6 Requirements of Barometer Signal to PSP Library

The barometer signal to the PSP library should meet the requirement as in Table 7.

Parameter	Requirement
Sampling Rate	4 Hz
Dynamic Range	70,000 to 135,535 Pa
Resolution	16 bits unsigned, 1 Pa/unit (70,000 to 135,535)
Sensor Noise	< 1.5 Pa RMS

Table 7 : Library Input Requirements for Barometer

Note that the dynamic range starts from 70,000 Pa. This is the offset that has to be taken into account when converting from the raw sensor signal to the signal input to the PSP library. In other words, 0x01 means 70,000+1 Pa (i.e. 70,001 Pa).

2.2 Host Microcontroller Requirements

The Host Microcontroller (MCU) of a wearable device is the container of PSP library. The library is designed to support 32-bit platforms, not limited for MCUs in wearable devices. The library is supplied as a pre-built binary library for a specific, predetermined ROM memory area for embedding into the host MCU. The C compatible functions are exposed to the host MCU for feeding inputs, processing metrics and collecting result metrics.

The software of Host MCU composes different software modules and drivers of hardware peripherals to form a complete system. The configuration of a wearable device depicted in Fig 1 shows the key software modules of a host MCU. The system software, which is beyond scope of this guide, includes Operating System, kernels and bootloaders that coordinate operations of each software modules.

The driver modules are software modules required to interface with the hardware peripherals like Bluetooth, 3D Accelerometer and PPG Analog Front End. The application module controls the application flows, user interface and external connections, like Bluetooth connectivity.

With the PSP library and pseudo example codes provided (as-is) by Philips, the system integrators have to adapt the pseudo example codes and build the other software components to complete the whole system. For PPG signal, to give a balance on power used and robustness on skin color, an adaptive LED power and PPG gain control is adapted. An example code of PPG gain control loop is provided (as-is) by Philips. System integrators should integrate and implement the PPG gain control logic with the driver of PPG Analog Front End. The LED power and PPG signal can also be adjusted automatically if the automatic gain control is available on the Analog front-end and is enabled.

The drivers for serial interfaces (SPI or I2C), PPG Analog Front End and 3D Accelerometer etc are important in the system integration. They provide means to collect sensor data from users and then being fed into PSP library to generate output metrics.

The System Controller is the core in scheduling the operation of PSP library in the wearable device. It is responsible to initialise the system components through the drivers, allocates resource and initiates the memory instance of PSP library. It schedules the operations in feeding sensor data, calling the library to process input data and generating enabled metrics, providing the metrics for UI displaying or transmitting to external devices. It releases resources and terminates the library's memory instance after all.

2.3 PSP library Overview

The library provides output metrics, which are either extracted from its input metrics, or are propagated input metrics. For details of metrics definition, please refer to [SY1901] and [SY1902].

Before starting the metrics extraction functionality, a block of memory must be assigned by MCU for an instance of the library. This instance should be terminated after definitively ending the extraction operation. The instance maintains the state of the extraction processing, which is the result of all input metrics data that has been processed so far. For details, please refer to [SY1901].

The output metrics must be enabled to tell the library to extract results of the enabled metrics. The system integrators must provide the required sensor data to the library as indicated in the ListRequiredMetrics(). Some optional input metrics can be fed to the library as well for optional features but not listed in the ListRequiredMetrics(), refer to [SY1901] for details. In addition, refer to [DG1903] and [SY1901] for multi stream PPG signals feeding mechanism to PSP library. The library is called to process after feeding one set of all sensor data. Then, the library provides output of the enabled metrics for specified update interval. For example, it updates heart rate metrics output every second after processing sets of PPG & ACC samples.

The PSP library requires all required sensor signals to be synchronized. For instance, for metrics in Biosensing by PPG, PPG and ACC signals are synchronized and simultaneously fed to the library at a constant number of samples per second. The library processes one set input with 32 PPG samples and 32 ACC samples every second. The MCU application has to constantly provide synchronized 32 PPG and 32 ACC inputs every second.

2.4 Memory Requirements

The MCU platform integrating the PSP library must provide the required resources for integrating the library. By design, the library targets a variety of 32-bit and 64-bit execution platforms, like the ARM platforms in mobile devices or embedded devices.

The PSP library for embedded ARM is supplied as a pre-built binary library for a specific, predetermined memory area. The system has to reserve the programming ROM in predetermined memory area for loading the pre-built binary library. The size of the ROM area is depending on the set of metrics included.

The library instance maintains the state of the metrics extraction processing. The MCU has to reserve the required RAM for a library instance and initialise it with the library function. An instance is associated with one single person as the maintained data are person-specific. In case a device does need to be used by multiple persons alternatingly, this can be realized by multiple library instances (one per device user). Therefore, multiple blocks of RAM could be required for library instances.

All metrics, especially like Resting Heart Rate, Stress Level Skin Conductance etc, have the ability to adapt to specific person over time. The history is stored in the instance RAM allocated to the library during initialisation. If the RAM used is volatile memory, which means the memory will be erased after reset or power cycle, the history will be lost. If system integrator would like to keep the history after reset or power cycle, the instance memory needs to be saved to a non-volatile memory and restored after the reset or power cycle. For more details refer to section 3.2.

Other than the RAM reserved for library instances, some RAM is used as stack for the library functions. The size of stack required varies depending on the metrics enabled.

The table below indicates the system requirements needed to integrate the platform. The figures provided in the table may vary slightly for different build releases and for different platforms.

Parameter	Requirement					
Supported platforms	32 and 64 bit platforms such as ARM Cortex M series					
Tested Platform	ARM Cortex M4(F)					
Metric Set	Biosensing by PPG	Biosensing by EDA	Fall Detection by AGB			
			ACC only	ACC+BARO	ACC+GYRO	ACC+GYRO +BARO
ROM[#]	78Kbytes	21Kbytes	17Kbytes	18Kbytes	22Kbytes	24Kbytes
Data RAM^{#^}	23Kbytes	25Kbytes	25Kbytes	25Kbytes	43Kbytes	43Kbytes
NVM^{&}	23Kbytes	25Kbytes	25Kbytes	25Kbytes	43Kbytes	43Kbytes
Stack RAM[#]	3.5Kbytes	0.8Kbytes	0.7Kbytes	0.7Kbytes	1.7Kbytes	1.7Kbytes
MIPS^{#, @}	0.6 MIPS	1.5 MIPS	Peak 27.4 MIPS* Median: 0.2 MIPS	Peak 27.4 MIPS* Median: 0.2 MIPS	Peak 91.6 MIPS* Median: 0.2 MIPS	Peak 91.8 MIPS* Median: 0.2 MIPS
[#] ROM/RAM/MIPS based upon ARM Cortex M4F implementation and depends on the set of included metrics. [^] Data RAM size required is specified for one instance only. ^{&} If system integrators would like to maintain the history of the metrics, system must contain the size of NVM specified. [*] This is the peak instantaneous MIPS when suspicious fall detected. [@] The MIPS are estimated based on a particular set of sensor input data.						

Table 8 : Memory and MIPS Requirements

2.5 Sensor Signal Requirements for Metrics

The PSP library supports a variety of metrics which require different categorization of sensor signals as input. Typically, the required metrics (except the optional input metrics) for each output metric can be found in the ListRequiredMetrics() at each update interval. Some of them assume the sensor signals to be continuously provided to the library while some others not as indicated by the ListRequiredMetrics(). The table below shows a summary of typical level of sensor signals required for different metrics. For more details, please refer to [SY1901] which is leading.

Metrics	Update Rate (s)	PPG required	ACC required	SC Required	Gyro/Baro Required
Activity Count	1	-	continuous	-	-
Activity Type	1	-	continuous	-	-
Active Energy Expenditure	1	continuous	continuous	-	-
Cadence	1	-	continuous	-	-

Cognitive Zone	60	-	-	continuous	-
Compressed Acceleration	1	-	continuous	-	-
Compressed PPG	1	continuous	-	-	-
Fall Occurrence	1	-	continuous	-	continuous
Fitness Index	60	continuous	continuous	-	-
Heart rate	1	continuous	continuous	-	-
Heartbeat timestamp	1	continuous	continuous	-	-
Low Power AEE	60	6 to 15 sec per minute	continuous	-	-
Low Power Heart Rate	60	6 to 15 secs per each minute	continuous	-	-
Private Data	60	-	continuous	-	-
Respiration Rate	1	continuous	continuous	-	-
Resting heart rate	3600	1 minute per each 15 consecutive minutes of low activity	continuous	-	-
Skin proximity	1	continuous while on-skin; during movement while off-skin	continuous	continuous ¹	-
Sleep Stages	-	continuous	continuous		
Speed	1	-	continuous	-	-
SpO2	1	continuous	continuous	-	-
Stress Level Heart Rate	1	continuous	continuous	-	-
Stress Level Skin Conductance	1	-	-	continuous	-
VO2Max	60	continuous	continuous	-	-

Table 9 : Sensor Signal input requirement for metrics

If low power mode is required, integrator can choose which metrics to enable. For example, Cadence, Activity Type and Low Power Heart Rate metrics can be enabled under low power mode in which the PPG channels are turned off most of the time.

¹ Skin proximity will use either PPG and ACC or Skin Conductance to determine skin proximity depending on which output metrics are enabled. Please refer to [SY1901] for Skin proximity extraction mechanism.

For all those metrics requiring continuous PPG signal, due to acquisition time and tracking time, turning them on and off frequently may reduce the measurement accuracy. System Integrators should design her own use case and validate the results.

CoreKinect LLC

3 Generic Guidelines for Integrating PSP library

3.1 Software Integration Flow

A software integration flow is suggested below which addresses data resampling and synchronizing required sensor signal (e.g. synchronization between PPG/ACC input signals), loading pre-built binary library and verifying the library function calls.

To make a product with good performance, the quality of sensor signals provided by the system to the library play predominant role. From the choice of components, design of circuits, design of mechanics and optical design, all these factors will directly impact sensor signal quality. Please consult the components supplier for state of the art mechanical and/or optical design. It is also recommended the system integrators to facilitate means in logging the raw sensor signals before feeding into the PSP library and propagated sensor signals from the PSP library for data analysis and trouble shooting.

Hence, the suggested design flow for the PSP library integration is as follows:

- Design hardware sensors connection scheme to MCU
- Design the methodology for sensors sampling and synchronization
- Design memory footprint for RAM, ROM, NVM, stack etc
- Prepare drivers to control and retrieve the hardware sensor data
- Implement data conversion to required rate and data synchronization, e.g. PPG and ACC data synchronization.
- Gain control for sensor signal, e.g. LED power and Photodiode gain control.
- Validate the input sensor data to see if it is meeting PSP library requirement.
- Build data logging (in preferred format as listed in section 3.5) capability to platform for debugging and development purpose
- Integrate PSP library to platform
 - o Include header and helper files
 - o Build application code to control lifecycle of the library
 - o Feeding data to library
 - o Extract output from library

3.2 Operation Flow of PSP library

3.2.1 Life Cycle Control - Fresh start

For typical initialization and calling control of the PSP library, please refer to [SY1901].

Below is a summary of the flow control of fresh start/fresh restart (without maintaining any previous history) (extracted from [SY1901]).

```

"allocate static memory" // allocate static memory for the instance,
                        // int32 should be used align to 32bit address

GetDefaultParams(...) // Get default parameters as reported by the library
"assign static memory and size to library" // Set static memory address and size to the library

"instantiate PSP"      // deployment format specific
...                   // state = Idle

repeat
  if ( " any metrics need to be enabled " )
    EnableMetrics(...) // enable metrics
  end if
  if ( " any metrics need to be disabled " )
    DisableMetrics(...) // disable metrics
  end if
  noEnabledMetrics = ( " no metric enabled "? )
  ListRequiredMetrics ( &requiredMetricsList )
  for ( each metricID in requiredMetricsList ) // PPG, acceleration, ...
    "gather metricData" // exactly 1 value per required metric
    SetMetric(metricID, metricData )
  end for
  Process ( ) // let PSP process anything it can
  ListUpdatedMetrics ( &updatedMetricsList )
  for ( each metricID in updatedMetricsList )
    GetMetric( metricID, &metricData )
    "interpret metricData"
  end for
until ( noEnabledMetrics      &&
      ( requiredMetricsList empty ) &&
      ( updatedMetricsList empty ) )

... // state = Idle
"terminate PSP" // deployment format specific

```

Fig 2. Operation flow of PSP library (fresh start/ fresh restart)

3.2.2 Life Cycle Control - Restart maintaining history

Metrics may accumulate historical data to adapt to a person and these data is stored in the RAM allocated. If the historical data is preferred to maintain after reset/power cycle, the system has to save the instance memory to the NVM first and then restore after reset/power cycle.

The following flow is recommended to maintain the memory during reset/power cycle.

1. Stop feeding data to library
2. Disable the metrics which require reset. Do not disable the metrics which require to keep the history except those metrics the history will still be kept while disabled, e.g. RHR for 7 days.
3. Save the instance RAM allocated, for instance plnst and a list of currently enabled metrics to NVM
4. Write a “yes” value (e.g. 0xAB23EF88, must be a very unique value and should not be easily hit by random memory content after device reboot) to a variable (e.g. keep_history) in NVM. This variable will remember if this is a fresh start or fresh restart or if it is intended to restore previous history state.
5. Reset/power cycle the device
6. Check keep_history variable. If detected “yes” value (e.g. 0xAB23EF88), restore saved memory from NVM to RAM location allocated, set plnst to previous saved value from NVM and retrieve the previous enabled metric list from NVM.
7. Set new TIME metric (with current time) to indicate time jump of reboot.
8. Write a “no” value to the variable (e.g. keep_history) in NVM to clear.
9. Continue normal process as usual.
10. Enable other metrics as usual.

Normally in the application code, the pspmem array (memory allocated for the instance) is defined as a static variable so that the address of it is, at each reboot, the same.

So copying data from NVM to this location will end up at the exact same position. This is important because all the internal parameters of the library are saved on the memory allocated.

The pseudo code should look like the following:

```

“allocate static memory” // allocate static memory for the instance,
                        // int32 should be used align to 32bit address

GetDefaultParams(...) // Get default parameters as reported by the library
“assign static memory and size to library” // Set static memory address and size to the library

if ( “keep_history = yes” ) // Check if restore NVM required
    “restore memory in NVM to instance memory”
    “restore pInst in NVM to instance pInst”
    “restore enabled metric list in NVM”
    SetMetric( pInst, PSP_METRIC_ID_TIME, ... ); // set new TIME metric to indicate time jump of reboot
else
    “instantiate PSP” // deployment format specific
end if
... // state = Idle

repeat
    if ( “ user indicates power reset and want to keep state?” )
        “set reset_powercycle to true” // Set flag to indicate reset requested and history keep
    end if
    if ( “ any metrics need to be enabled ” )
        EnableMetrics(...) // enable metrics
    end if
    if ( “ any metrics need to be disabled ” )
        DisableMetrics(...) // disable metrics
    end if
    noEnabledMetrics = ( “ no metric enabled ”? )
    ListRequiredMetrics ( &requiredMetricsList )
    for ( each metricID in requiredMetricsList ) // PPG, acceleration, ...
        “gather metricData” // exactly 1 value per required metric
        SetMetric(metricID, metricData )
    end for
    Process ( ) // let PSP process anything it can
    ListUpdatedMetrics ( &updatedMetricsList )
    for ( each metricID in updatedMetricsList )
        GetMetric( metricID, &metricData )
        “interpret metricData”
    end for
until ( ( reset_powercycle ) || ( ( noEnabledMetrics ) &&
    ( requiredMetricsList empty ) &&
    ( updatedMetricsList empty ) ) )

if (“keep history after reset or power cycle”) // Check if history is maintained after reset
    “write yes to keep_history in NVM ”
    “disable those metrics not intended to keep history”
    “Save the instance RAM allocated and pInst to NVM”

```

```

    "Save enabled metric list to NVM"
    "power down or reset the device"
endif
...
"terminate PSP" // state = Idle
                // deployment format specific
    
```

Fig 3. Operation flow of PSP library (restart to maintain the history)

3.2.3 Calling Protocol Description

Before starting the metrics extraction, the application creates an instance by allocating memory of the minimum size indicated by `GetDefaultParams` or bigger, and passing it to `Initialise` function. The memory for the library instance must be provided by the application. The library instance maintains the state of the metrics extraction, which actually is the result of all input metrics data that the library has processed so far. During the entire lifecycle of the instance, the application maintains and should not modify the instance memory. The application passes the instance identifier (got at initialization) of the instance with any function calls to the library. At the end of the lifecycle `Terminate` should be called after the definitive end of the metrics extraction to release the resources got at initialization.

The application must specify, via `EnableMetrics`, which output metrics it wants the library to provide. Likewise, the application can also disable output metrics again. The application can enable or disable metrics together or one by one over time with application needs.

While the library is extracting output metrics, `ListRequiredMetrics` indicates which mandatory input metrics are required in relation to the enabled output metrics. It is the responsibility of the application to constantly provide any required input metrics and to call `Process` so that the extraction of output metrics can take place.

After each `Process`, the library indicates, via `ListUpdatedMetrics`, which metrics have been updated, so that the application may selectively get the updated metrics via `GetMetric`. It is not mandatory to get updated metrics; however, the metric value may be updated (i.e. overwritten) in a subsequent `Process`.

The application is not bound to get only updated metrics; it may get any metric at any time, yielding the last updated value. As long as the value is not updated, multiple gets will result in the same metric value being repeatedly provided.

The list of required input metrics may actually vary dynamically over time: PSP may e.g. only need PPG data once in a while to extract a given metric. By this, PSP allows the application to minimize resource usage. It enables the application to switch off sensor hardware for the period that a metric sensed by that hardware is not required, so as to minimize power usage. For example, Low Power

Heart Rate only requires a few seconds of PPG signal per minute, the PPG front ends can be switched off for the remaining periods.

The updated metrics have its own data representation in the output syntax. The MCU program should extract the metrics value based on the syntax of output metrics as indicated in [SY1902] and provide those values for user display or further processing.

The functions of the library should be called one after one. The functions are not designed for any parallel operation. For which functions should be called under which states, please refer to [SY1901] section 3.

3.3 PSP API Function Usage Example

3.3.1 PSP_GetVersionInfo()

PSP_GetVersionInfo() provides information to identify the version of library. A memory array of 64 bytes at least (or even bigger depending on the actual returned size) must be provided by the system. This function can be invoked before any library is instantiated. It is useful to determine if a correct version of library is running or an update of the library is required.

PSP_GetVersionInfo() fills an array of byte data with version information that is unique for each version of the library. The version information may be used to implement additional functions in system initialisation or leave empty if not required on the target platform.

Example of calling PSP_GetVersionInfo() is given below.

```
/* Example of function call */
/* VERSION INFO ***** */
FX_UINT08 versionInfo[64]; // declaration used in general
FX_UINT16 versionSize = sizeof(versionInfo);

status = PSP_GetVersionInfo( versionInfo, &versionSize );

/* Example of returned parameters */
/* versionInfo = [01 02 03 30 2D 31 32 33 34 35 36 37 20 20 20 20 FD FF BF FF] */
/* main version = 01 */
/* major revision = 02 */
/* minor revision = 03 */
/* internal configuration = 30 2D 31 32 33 34 35 36 37 20 20 20 20 FD FF BF FF */
```

3.3.2 Library Instance Parameters

System integrator can specify a number of parameters, e.g. memory allocated for the library instance, in the PSP_INST_PARAMS. pMem points to the address of allocated memory and memorySize specifies instance memory size. The maximum data RAM required for full metric library can be found in Table 8.

The type structure for the library instance parameters is defined in `psp.h`.

```
/** @brief PSP Instance Parameter structure type. */
typedef struct psp_inst_params_tag
{
    FX_VOID * pMem;          /**< Pointer to memory block used for instance initialisation */
    FX_UINT08 * pSourceID;    /**< Pointer to memory block containing the SourceID */
    FX_UINT32 memorySize;     /**< Size of memory block appointed by pMem field */
    FX_UINT16 sourceIDSize;   /**< Size of memory block appointed by pSourceID field */
    FX_SINT16 calCoefSpO2[3]; /**< array of SpO2 calibration coefficients: a, b, c , */
                                /** scaled with factor 100 */
    FX_UINT08 alarmRateFall;  /**< false alarm rate for fall: 0: once per day, 1: once */
                                /** per week, 2: once per month */

} PSP_INST_PARAMS, * PPSP_INST_PARAMS;
```

Array `calCoefSpO2[3]` is used to pass the SpO2 calibration parameters to the library to calculate SpO2. These parameters are form factor and sensor specific. So, each device is supposed to have a specific set of parameters after calibration and should be set before library initialisation.

Parameter `alarmRateFall` is used to adjust the false alarm rate for Fall Detection. Also, should be set before library initialisation.

3.3.3 PSP_GetDefaultParams()

`PSP_GetDefaultParams()` provides default parameters that can be used in initializing an instance of a library by `PSP_Initialise()` function. The application should allocate memory of the minimum size indicated by `memorySize` returned from `PSP_GetDefaultParams()` or bigger, and then pass to the `PSP_Initialise()` function allocated memory pointing by `pMem`.

3.3.4 PSP_Initialise()

Example of the memory allocation and initialization of an instance is given below.

```
#define PSP_MEMORY_SIZE ( 50000 )

static  PPSP_INST      sInst;
uint32_t pspmem[PSP_MEMORY_SIZE/4];

/* INITIALISATION ***** */
PSP_ERROR pspError = PSP_ERROR_NONE;
PSP_INST_PARAMS params;

(void)PSP_GetDefaultParams (&params);

params.memorySize = sizeof(pspmem);
params.pMem = pspmem;
params.pSourceID = (FX_UINT08*) "ABCD123"; //User specified, e.g. "ABCD123"
params.sourceIDSize = 8; //equals to length of pSourceID + 1, e.g. 8
/* Some other parameters can be set as well, like SpO2, Fall Occurrence */

/* Initialize PSP library */
pspError = PSP_Initialise(&params, &sInst);
// "actual memory used/required is returned as : params.memorySize"
...
```

The pSourceID in the parameter for instance initialisation is used to identify the person-specific data. It must be set to 8-byte long unique identifier (including null character) to show the data source for Sleep Algorithm. Otherwise, pSourceID should be set to null with sourceIDSize = 0.

Note that the allocated memory for the instance MUST be aligned to boundary of 32-bit words. So, a number of 32-bit words, pspmem[PSP_MEMORY_SIZE/4], are allocated.

The PSP_GetDefaultParams() function helps to determine the size of memory required for an instance before the memory is allocated.

PSP_Initialise() creates an instance with memory provided by the host MCU. This function initialises the RAM resources based on the information provided to the function. The parameter set for this instance cannot be modified during the lifetime of the instance. An instance is associated with one single person as the maintained data are person-specific. In case a device does need to be used by multiple persons alternately, this can be realized by multiple library instances (one per device user) with unique identifier, i.e. pSourceID specified.

3.3.5 PSP_EnableMetrics()

PSP_EnableMetrics() enables extraction of the available metrics. The available output metrics must be enabled to tell the library to extract results of the input metrics.

An example of calling PSP_EnableMetrics() with enabling Heart Rate and Skin Proximity metrics is given below.

```
/* ENABLE METRICS ***** */
PSP_METRIC_ID metricIdList[] = { PSP_METRIC_ID_HEARTRATE,
                                PSP_METRIC_ID_SKINPROXIMITY
                                };
status = PSP_EnableMetrics( pPspInst, sizeof( metricIdList ), metricIdList );
```

3.3.6 PSP_ListRequiredMetrics()

PSP_ListRequiredMetrics() checks which of the input metrics must be required. May also refer to Table 4 of [SY1901] to conceptually know the required input metrics for each enabled output metrics. Practically, in runtime, the output of PSP_ListRequiredMetrics() is the right one need to be honored.

Example of calling PSP_ListRequiredMetrics() to check required input metrics is given below.

```
/* GET AND FEED REQUIRED INPUT METRICS ( ACC, PPG, others ) ***** */
PSP_METRIC_ID requiredMetrics[10];
FX_UINT08 numberOfRequiredMetrics;

numberOfRequiredMetrics = sizeof(requiredMetrics);

status = PSP_ListRequiredMetrics( pPspInst, requiredMetrics, &numberOfRequiredMetrics );

/* Example of returned parameters */
/* numberOfRequiredMetrics = 0x01 */
/* requiredMetrics = [0x2B] */
```

3.3.7 PSP_SetMetric()

PSP_SetMetric() feeds the input metric data, e.g. PPG, ACC and other inputs required for PSP library.

An example of feeding data to PPG Green metric is given below.

```

/* FEED 32 PPG SAMPLES IN EACH CYCLE ##### */
#define PSP_NUMBER_OF_PPG_SAMPLES    (32);

/* add PPG data in ppg from buffer */
/* ppg[PSP_NUMBER_OF_PPG_SAMPLES] */
/* buf_ppg : ppg buffer from hardware */
for ( int j = 0; j < PSP_NUMBER_OF_PPG_SAMPLES; j++ )
{
    ppg[j] = buf_ppg[j]; /* Copy PPG signal from hardware buffer to input buffer */
}

/* fill ppg metric array */
index = 0;
metricId = PSP_METRIC_ID_PPG_GREEN;
/* metricSize: IX + Q + BPI + PF + SI + OFS + EXP + Li (0 - 3) + Gi (0 - 3) + P_LPH (0 - 32) */
/* 15 + 2 * 32 = 79 */
/* Since EXP is optional byte, if EXP is not used, the size is reduced by one */
/* metricSize: IX + Q + BPI + PF + SI + OFS + Li (0 - 3) + Gi (0 - 3) + P_LPH (0 - 32) */
/* 15 + 2 * 32 = 78 */

metricSize = ( PSP_NUMBER_OF_PPG_SAMPLES * 2 ) + 15;
data[index++] = PSP_METRIC_ID_PPG_GREEN;
data[index++] = (uint8_t)( metricSize & 0xFF );
data[index++] = (uint8_t)(( metricSize >> 8 ) & 0xFF );
data[index++] = sequence;          // IX for feed arbitrary number, increment by 1 each
                                   // time and wrap around 255. 0 to 255.
data[index++] = 4;                 // Quality(Q), set fixed to 4 for input metric
data[index++] = bodyPosition;      // BPI, fill in sensor wrist position
data[index++] = 0x60;              // PF, fixed to 0x60 for 32 PPG samples
data[index++] = 0x03;              // SI, Stream locations identifier, e.g. 0x03
data[index++] = 0x00;              // OFS, PPG Offset, set to zero, not yet supported
data[index++] = 0x00;              // EXP, PPG Exponent, set to zero, not yet supported
                                   // Do not need to set this byte if total size is 78
data[index++] = ledPower[0];        // L0, ledPower value [0..100] for 1st quarter of second
data[index++] = ledPower[1];        // L1, ledPower value [0..100] for 2nd quarter of second
data[index++] = ledPower[2];        // L2, ledPower value [0..100] for 3rd quarter of second
data[index++] = ledPower[3];        // L3, ledPower value [0..100] for 4th quarter of second
data[index++] = adcGain[0];         // G0, ADC Gain value [0..3] for 1st quarter of second
data[index++] = adcGain[1];         // G1, ADC Gain value [0..3] for 2nd quarter of second
data[index++] = adcGain[2];         // G2, ADC Gain value [0..3] for 3rd quarter of second
data[index++] = adcGain[3];         // G3, ADC Gain value [0..3] for 4th quarter of second
for ( int sample = 0; sample < PSP_NUMBER_OF_PPG_SAMPLES; sample++ )
{
    data[index++] = (uint8_t)( ppg[sample] & 0xFF );          // PPG - LSB
    data[index++] = (uint8_t)(( ppg[sample] >> 8 ) & 0xFF );  // PPG - MSB
}
/* transfer ppg metric to library */
status = PSP_SetMetric( pPspInst, PSP_METRIC_ID_PPG_GREEN, data, index );

```

An example of feeding ACC data to Acceleration metric is given below.

```
/* FEED 32 ACC SAMPLES PER 1 SECOND */
#define PSP_NUMBER_OF_ACC_SAMPLES    (32);

/* add ACC data in acc from buffered buf_acc[3][32] */
for ( int i = 0; i < PSP_NUMBER_OF_ACC_SAMPLES; i++ )
{
    for (int x=0;x<3;x++)
    {
        acc[x][i] = buf_acc[x][i]; /* Copy ACC signal from hw buffer to input buffer */
    }
}

/* fill acc metric array */
index = 0;
metricId = PSP_METRIC_ID_ACCELERATION;
metricSize = ( PSP_NUMBER_OF_ACC_SAMPLES * 6 ) + 4;    // ( number of samples * 6 ) + index +
               Q + BP + AF
data[index++] = PSP_METRIC_ID_ACCELERATION;
data[index++] = (uint8_t)( metricSize & 0xFF );
data[index++] = (uint8_t)(( metricSize >> 8 ) & 0xFF );
data[index++] = sequence;          // IX for feed arbitrary number, increment by 1 each
                                   // time and wrap around 255. 0 to 255.
data[index++] = 4;                 // Quality(Q), set fixed to 4
data[index++] = bodyPosition;      // BFI, fill in sensor wrist position
data[index++] = 0x6E;              // AF, set to 0x6E for 32 ACC samples
for ( sample = 0; sample < PSP_NUMBER_OF_ACC_SAMPLES; sample++ )
{
    data[index++] = (uint8_t)( acc[0][sample] & 0xFF );          // X - LSB
    data[index++] = (uint8_t)(( acc[0][sample] >> 8 ) & 0xFF ); // X - MSB
    data[index++] = (uint8_t)( acc[1][sample] & 0xFF );          // Y - LSB
    data[index++] = (uint8_t)(( acc[1][sample] >> 8 ) & 0xFF ); // Y - MSB
    data[index++] = (uint8_t)( acc[2][sample] & 0xFF );          // Z - LSB
    data[index++] = (uint8_t)(( acc[2][sample] >> 8 ) & 0xFF ); // Z - MSB
}
/* transfer acc metric to library */
status = PSP_SetMetric( pPspInst, metricId, data, index );
```

3.3.8 PSP_Process()

PSP_Process performs the processing of the input metrics and produce the output metrics.

```
// The minimum number of cycle refers to [SY1901]
while (#16 cycles per second)
{
    // Feed input metrics if requested

    /* DO PROCESSING ##### */
    status = PSP_Process( pPspInst );
}
```

3.3.9 PSP_ListUpdatedMetrics()

PSP_ListUpdatedMetrics() checks which of the metrics are updated.

An example of calling PSP_ListUpdatedMetrics() is given below.

```
/* get updated metrics list */
PSP_METRIC_ID metrics[32]; // size must be big enough for metrics enabled
FX_UINT08 numberOfMetrics;

numberOfMetrics = sizeof(metrics);

status = PSP_ListUpdatedMetrics( pPspInst, metrics, &numberOfMetrics );

/* Example of returned parameters */
/* numberOfMetrics = 0x02 */
/* metrics = [0x20 0x22] */
```

Note that the size of metrics must be big enough for the number of metrics enabled.

CoreKinect LLC

3.3.10 PSP_GetMetric()

PSP_GetMetric() gets the metric data. The same method illustrated in following example for Heart Rate metric is applicable to other metrics.

```
status = PSP_ListUpdatedMetrics( pPspInst, metrics, &numberOfMetrics );

for ( int k = 0; k < numberOfMetrics; k++ )
{
    /* Get updated output metric values */
    FX_UINT08    metricData[PSP_MAX_METRIC_SIZE];
    FX_UINT16    metricDataSize;

    metricDataSize = sizeof(metricData);

    status = PSP_GetMetric( pPspInst, metrics[k], metricData, &metricDataSize );

    if ( status == PSP_ERROR_NONE )
    {
        /* interpret metrics */
        switch ( metricData[0] )
        {
            case PSP_METRIC_ID_HEARTRATE:
                /* interpret Heart Rate */
                metricSize      = metricData[1];
                metricSize |= metricData[2] << 8;
                metricIndex     = metricData[3];
                metricQuality    = metricData[4];
                metricHeartrate = metricData[5];
                break;

            case PSP_METRIC_ID_SKINPROXIMITY:
                /* interpret Skin Proximity */
                metricSize      = metricData[1];
                metricSize |= metricData[2] << 8;
                metricIndex     = metricData[3];
                metricQuality    = metricData[4];
                metricSkinProx   = metricData[5];
                break;

            default:
                break;
        }
    }
}
```

3.3.11 PSP_DisableMetrics()

PSP_DisableMetrics() disables the metrics extraction.

An example of calling PSP_DisableMetrics() with disabling Heart Rate metric is given below.

```
/* DISABLE METRICS ***** */
PSP_METRIC_ID metricIdList[] = { PSP_METRIC_ID_HEARTRATE
};

status = PSP_DisableMetrics( pPspInst, sizeof( metricIdList ), metricIdList );
```

3.3.12 PSP_Terminate()

PSP_Terminate() cleans up the memory resources used by the PSP instance. The host system is responsible to manage or release the allocated resources of the instance.

```
/* INITIALISATION ***** */
pspError = PSP_Terminate(&pspInst);
```

3.4 Sensor Driver

System integrators are responsible to implement the driver for the sensors and deliver the expected sensor signals to the PSP library.

3.4.1 Driver for PPG front End

The PPG Front End is designed to measure optical signals from reflected LED pulses and ambient light. The PPG Analog Front End should perform cancellation of ambient light interference to get a digital converted PPG signals which contain variation caused by blood perfusion on the wrist. The PPG Analog Front End also contains configurable LED drivers in which the power output is controlled by the host MCU.

The driver for the PPG Analog Front End is responsible to initialise and configure the PPG Analog Front End to acquire the PPG samples in pre-determined frequency, set the power of the LEDs and conversion gain of photodiode pickups. Accompanying with the serial communication interface, either SPI or I2C, the driver is able to send the control commands and get the measured values of PPG signals in a multi-bytes packages compliant with the FIFO and data format of PPG Analog Front End.

The FIFO Almost Full (Watermark) interrupt can be enabled. Then the PPG Front End will send an interrupt to the MCU when desired number of PPG samples are acquired. Then, the FIFO is read to get the acquired PPG samples.

The activity on the serial interface pins can introduce noise on the on-chip GND potential, disturbing an optical sample, resulting in higher noise. Therefore, during a FIFO read event, it is recommended to schedule the FIFO read to occur between optical samples. This can be accomplished by reading the FIFO when the watermark interrupt occurs and then limit the number of samples in the FIFO to those that can be read out during the time between samples, depending on the serial interface transfer speed.

In the interrupt process of PPG, the number of ACC samples acquired should be determined and the ACC samples will be read from the accelerometer in subsequent process. The idea is to let the PPG clock as the reference and avoid resampling PPG signals as little as possible.

3.4.2 Driver for Accelerometer

The accelerometer is designed to measure the motion of end users. It is also used to compensate the motion artifacts for heart rate calculation. It is recommended to make use of the on-chip FIFO to accumulate the ACC samples for every second. The ACC data transmission from accelerometer to host MCU occurs once per second in order to minimize the time and power consumption for data transfer. A memory buffer for accelerometer data is used if the size of on-chip FIFO is not enough to contain the ACC samples in one second.

The driver for the accelerometer is responsible to initialise and configure the accelerometer to acquire the ACC samples in pre-determined frequency. Accompanying with the serial communication interface, either SPI or I2C, the driver is able to send the control commands and get the measured values of ACC signals in a multi-bytes packages.

3.4.2.1 ACC Sensor Orientation Adjustment

The acceleration signal consists of 3 axes: X, Y and Z. It is important that these axes have a certain relation to the casing of the embodiment. For PSP library, the axes should be aligned in case of a watch embodiments such that the time can be read:

- The X axis points upwards ("12 o'clock") with 1g reading on X-axis
- The Y axis points upwards ("9 o'clock") with 1g reading on Y-axis
- The Z axis points towards the nose of the reader with 1g on Z-axis

The orientation is illustrated as below.



Fig 4. The orientation of accelerometer for Philips VitalSigns Optical Library

A mapping could be used to get the correct orientation in feeding ACC signals to PSP library. This mapping can be done in the ACC driver.

One can follow steps below to determine the mapping of accelerator axes orientation:
First, put the clock with 12 o'clock upwards, i.e. the clock hand points to 12 and point to sky. Read the acceleration signal to find axis and orientation with 1g output. The acceleration signal should map to the x-axis of feeding ACC samples.

Then, put the clock with 9 o'clock upwards, i.e. the clock hand points to 9 and point to sky. Read the acceleration signal to find axis and orientation with 1g output. The acceleration signal should map to the y-axis of feeding ACC samples.

Finally, put the clock with the watch facing upwards, read the acceleration signal to find axis and orientation with 1g output. The acceleration signal should map to the z-axis of feeding ACC samples.

3.5 Data Logging

In the integration phase, raw data from sensor drivers, input sensor data to PSP library and output data (including the propagated data) from PSP library are very important in validation and are suggested to log into a file for analysis.

The data format and folder structure should follow the guidelines below:

- CSV format. The first row indicates the column definition. Each row afterward represents one individual sampling point for all those data to be captured. It is preferred to log the timestamp in ms unit in the first column.
- Each sensor signal or PSP input/output metrics must be captured in an individual file.
- The file path, file name and file content must not include any personal identity.
- For the full path, folder name and file name
 - o Each recording must be stored in an individual folder.
 - o Only English and number are allowed in the naming. No space nor special character nor Chinese etc are allowed in the file or file/path name.
 - o The full path should be as short as possible to avoid redundant information.
 - o Log file name must have the same prefix and file type, e.g. ppg_XXXXX.csv
- In the CSV file, number must be in number format, should not be string.
- The PPG and ACC signals could be the PSP propagated or the raw driver signals. However, it is better to log both such that the conversion logic from the raw driver signals to the PSP input could also be checked.
 - o If the PPG and ACC logged signal are coming from raw driver and the sampling is not ideal/stable, accurate timestamps must be captured in file for resampling purposes later.
 - o PPG and ACC signal must be synchronized. If not, must have the accurate timestamps for each raw signal.
 - o The data format, resolution and range must meet the requirement of PSP library and made available to Philips.
- To compare to ground truth, those ground truth reference must be in a separated file placed under the same folder as the recording. All ground truth references must have the same prefix and should be in CSV format, e.g. ref_hr_XXXX.csv. The timeline of the reference log and raw signal log are not necessary to be already synchronized. If they are not synchronized, Philips can do the synchronization during data processing. However, if they are not

synchronized, the time difference cannot be too much. For the ground truth of SpO2, the reference SpO2 device need to deliver each SpO2 value along with an HR value.

- The logged data should be sent to Philips by the agreed channel.

If there is doubt about the CSV format, please contact Philips ITS Customer Support team for guidance.

After Philips receives the CSV format, those files should be converted to Philips internal FT format for offline processing.

Here is an example of PPG and ACC in FT format.

[illegible]

Fig 5. Example of ACC data in FT format

```
% date : 2016-03-03   time : 09:42:02
% Format ID : 258
% OHR Firmware version : 5.3.3
% Raw Version Info : 01-00-00-00-00-c2-01-00-01-00-00-01-02-00-00-00-01-00-ec-fe-14-01-3c-58
% Interval : 1 s
% File source : using FX Library, v6.4.2_20210211_1140
% Body Position : 1
% Fs : 32 Hz
% LED Color: Green
% LED radius: 0 mm
% LED position: 0
% PD radius: 0 mm
% PD position: 0
% quality  adc_gain[0..3]  led_power[0..3]  offset  exponent  ppg[0..31]
4   1   2   2   2   10  12  12  12   0   0 16870 16868 16870 16870 16874 16874 16864 16860 35630 36900 37744 38304 38682 38934 39102 39224 39318 ...
4   2   2   2   2   12  12  12  12   0   0 42286 42430 42406 42180 41814 41406 41012 40648 40332 40118 39940 39806 39710 39636 39574 39542 39534 ...
4   2   2   2   2   12  12  12  12   0   0 41340 41276 41200 41140 41054 40978 40856 40710 40538 40392 40282 40228 40192 40242 40292 40272 40192 ...
4   2   2   2   2   12  12  12  12   0   0 37640 37234 36934 36706 36562 36430 36380 36352 36342 36362 36410 36478 36546 36650 36772 36892 36988 ...
```

Fig 6. Example of PPG data in FT format

The output metrics collected from the PSP library with `PSP_GetMetric()` could also be captured in FT format to provide a way for performance validation.

[illegible]

% quality	heartrate
0	71
2	60
3	58

Fig 7. Example of logging heart rate output in FT format

Note the format ID and the additional header information associated with each recording for debugging purposes. If System Integrators can directly log the data into FT format, it also works for Philips but it is not strictly required.

The FT example format show that the logged information is captured from the raw data sending to the PSP library with `PSP_SetMetric()`, propagated sensor and information getting from the PSP library with functions, `PSP_GetMetric()` and `PSP_GetVersionInfo()`.

To implement the data logging, the system should save the data in the system memory or sent the data through Bluetooth or other wireless link. Same approach can be implemented by system integrators in their system, however, the size of data storage or the data transmission rate should be taken into account. It requires about 20.5K bytes of memory to store data for a minute or 0.5M bytes for an hour. The transmission rate should up to 3.5K bit per second through either wired or wireless links.

In some of Philips legacy evaluation devices, binary file format 'WeST Interchangeable File Format' (WIFF) as described in [PRE13-13-583] to store the data file in human unreadable format was used.

3.5.1 Legacy WIFF binary format Overview

The 'WeST Interchangeable File Format' (WIFF) is a proprietary, binary file format designed for storing personal health data. The file format is used in Philips reference sample for data logging.

Please refer to [PRE13-13-583] for details of the WIFF format.

Below is an example of WIFF file content. Note the container boundaries are yellow highlighted part. The red highlighted parts are metrics IDs.

00000000	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f	
00000000	4f	48	52	11	00	0a	62	53	1f	00	00	00	00	00	00	00	OHR...bS.....
00000010	00	00	e9	25	70	61	4f	48	52	30	00	0b	00	00	00	00	..é\$paOHR0.....
00000020	00	00	00	00	06	01	00	32	37	34	2d	33	38	34	33	34	274-38434
00000030	39	61	2d	72	15	04	02	06	01	00	32	37	34	2d	33	38	9a-r.....274-38
00000040	34	33	34	39	61	2d	72	ee	ff	ff	6c	4f	48	52	84	00	4349a-rîÿÿlOHR,,
00000050	0c	3f	3f	3f	02	03	00	32	33	38	2d	31	34	61	61	62	.???...238-14aab
00000060	39	31	2d	00	01	01	02	0c	03	11	00	25	00	04	50	48	91-.....\$.PH
00000070	47	50	35	38	20	08	02	41	c2	0f	41	70	64	03	10	08	GP58 ..AÂ.Apd...
00000080	21	4c	49	53	32	44	57	31	32	01	22	19	09	23	01	00	!LIS2DW12."...#..
00000090	00	00	01	00	00	00	01	08	41	4d	41	58	38	36	31	34	AMAX8614
000000a0	31	01	42	20	05	43	00	02	04	01	00	05	43	01	02	00	1.B .C.....C...
000000b0	01	00	1d	61	54	79	70	65	20	43	34	2e	30	28	33	32	...aType C4.0(32
000000c0	39	32	6d	56	29	20	2d	20	53	57	20	32	2e	31	2e	31	92mV) - SW 2.1.1
000000d0	39	01	62	19	4f	48	52	31	01	0f	03	03	00	01	04	00	9.b.OHR1.....
000000e0	04	08	00	01	04	ad	07	02	17	01	02	05	03	00	01	04	-.....
000000f0	af	06	04	00	01	04	02	03	09	03	00	01	04	00	1e	08	-.....
00000100	00	01	04	ea	25	70	61	04	04	20	03	00	01	00	00	22	...é\$pa.."
00000110	03	00	01	04	fe	23	04	00	01	00	00	00	24	03	00	01	b#.....\$...
00000120	00	00	25	03	00	01	00	00	26	03	00	01	00	01	27	04	..\$......&.....'.
00000130	00	01	04	00	02	2a	03	00	01	00	00	2b	c4	00	01	04	*.....+Ã...
00000140	02	6e	05	00	c9	ff	85	01	20	00	c1	ff	06	02	26	00	.n..Éÿ.... Áÿ...&.
00000150	c8	ff	12	02	0e	00	ca	ff	0a	02	21	00	d1	ff	0e	02	Èÿ....Èÿ...!..Ñÿ..

3.6 Library Deployment

In common tools chain, the linker options are used to reserve the code memory range for library and ensure the memory range is not used for any application code. Examples are given in following sections on how to load the binary library into the code memory.

First is to define the macro for the desired platform: e.g. #define FX_PLATFORM_ARM_M4, this will select the right data type definition in the fx_datatypes.h for the target platform.

3.6.1 Loading Binary library image

The executable binary library has to be put into the processor which means loading the binary into the predetermined memory area reserved for the library. The system integrators can load the binary library, psp.hex or psp.bin, into the code memory separately with a programmer. Alternatively, the binary library can be merged with the binary image of main program with tools, like srec_cat.exe, before loading the whole program into the code memory with a programmer.

The binary library image, psp.bin, can be incorporated into the application project. The procedure will be different for different development environments. Here below are steps required for Keil and

IAR to incorporate the binary library image into the application project. In both cases, some linker options are involved to reserve code memory range for the PSP library.

3.6.1.1 In Keil uVision5 environment

To incorporate the binary library image for Keil uVision5, an assembler file is created that contains data section with INCBIN that imports the PSP binary file, psp.bin. Then, controls the linker to load that data section into specific memory range by modifying the scatter file. In the meanwhile, the code memory range for the PSP library is also reserved for that data section.

Example in linking psp.bin binary in designed memory location (0x20000 – 0x2FFFF):

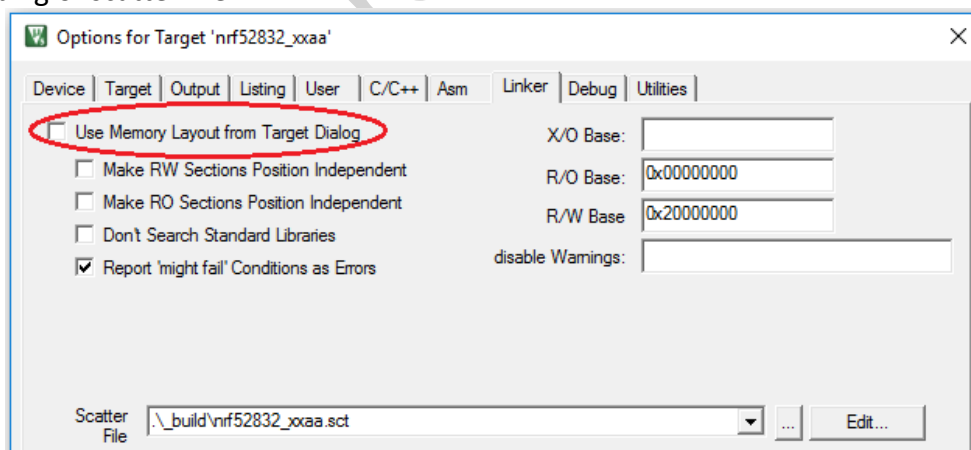
Step 1: Create an assembler file, psp_image.s, in the project containing a data section and import the binary using the INCBIN directive.

```
AREA psp_image, CODE, READONLY, PREINIT_ARRAY
INCBIN .\inc\psp.bin
END
```

Step 2: Modify the scatter file to place that data section into specific memory location.

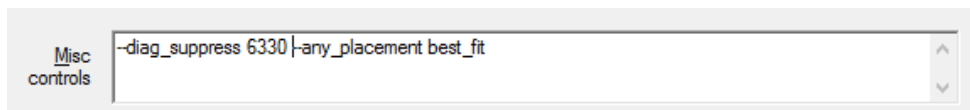
```
LR_ROM_0x00020000 0x00010000 { ; RO section for the PSP library
  ER_ROM_FX 0x00020000 0x00010000 { ; RO Data load address = execution address
    psp_image.o (+RO)
  }
}
```

Please uncheck the “Use Memory Layout from Target Dialog” option to enable memory mapping of scatter file.



Step 3: In order to get optimization on the code allocation, you have to add following options for the target

Add "--any_placement best_fit" to misc control field of the linker options.



Step 4: Verify the binary image, psp_image.o, is loaded with the memory mapping file. Following section should be found in the mapping file, like nrf52832_xxaa.map

```
Load Region LR_ROM_FX (Base: 0x00020000, Size: 0x0000e764, Max: 0x00010000, ABSOLUTE)
Execution Region ER_ROM_FX (Exec base: 0x00020000, Load base: 0x00020000, Size: 0x0000e764, Max: 0x00010000,
ABSOLUTE)
Exec Addr  Load Addr  Size      Type  Attr  Idx  E Section Name  Object
0x00020000  0x0000e762  Code RO    1665  psp_image      psp_image.o
```

Detail of the implementation is described in following reference.

<http://www.keil.com/support/docs/3770.htm>

<http://www.keil.com/forum/20886/> (see 5th message)

<http://www.keil.com/forum/18585/>

3.6.1.2 In IAR Embedded Workbench environment

To incorporate the binary library image for IAR Embedded Workbench, a string in a fixed memory area is defined and a feature in IAR Embedded Workbench for ARM is used that can link one binary file of any kind (e.g. bin, bmp, wav, etc.) into a project through the linker option, --image_input. The code memory range for PSP library is reserved under the definition of the string variable.

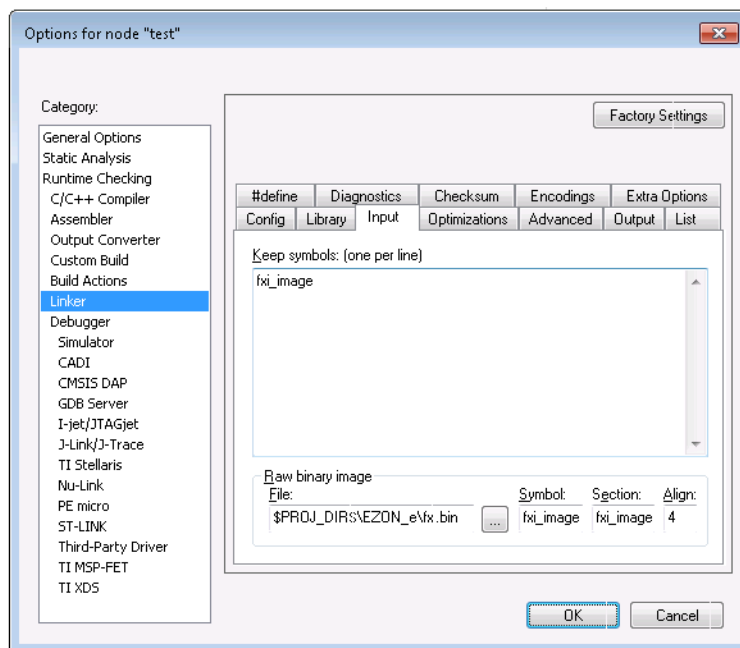
Example in linking psp.bin binary in designed memory location (0x20000 – 0x2FFFF):

Step 1: Define a string in a fixed flash address (e.g. address 0x020000)

```
extern const char * psp_image;

#pragma data_alignment=4
#pragma location=0x020000
static char psp_string[65536]; //no assigned value
```

Step 2: Assign a value to the string from a binary file (Set the linker options of IAR ILINK Linker) to include a binary image (psp.bin). You have to add a symbol and the filename in options. The option can be found as.

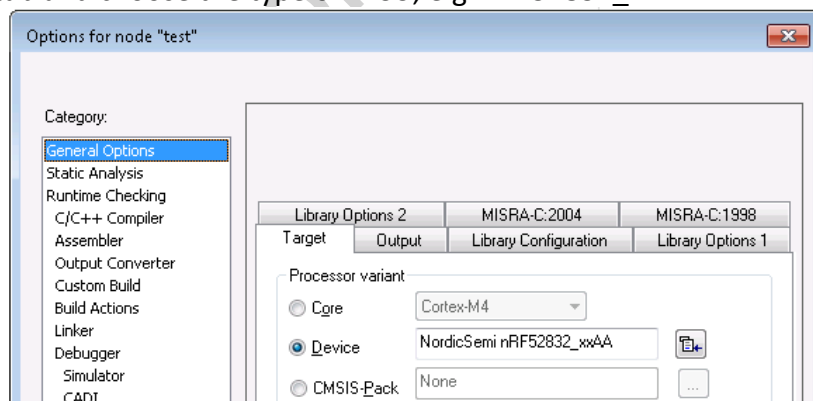


Add "\$PROJECT\$\psp.bin" as File, "psp_image" as Symbol, "psp_image_section" as Section and Align = 4 to input tag of the linker options.

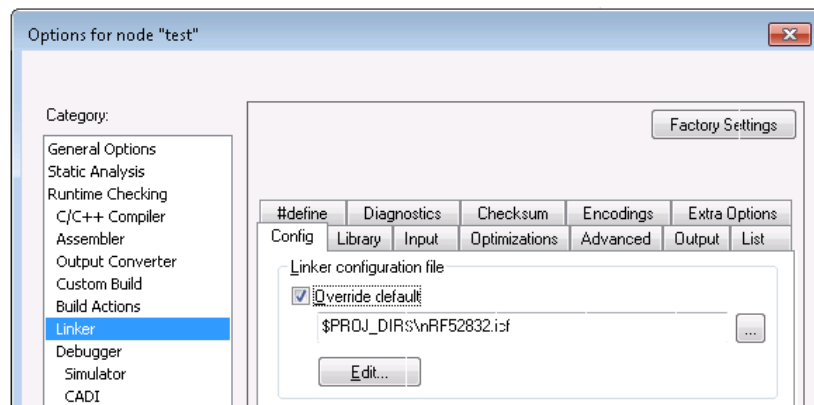
It applies option "--image_input=\$PROJECT\$\psp.bin, psp_image, psp_image_section, 4" to the linker.

Step 3: Place the binary image with a linker file directive:

Open the Options dialog from the menu Project -> Options. In the General Options page, go to the Target tab and choose the type of MCU, e.g. nRF52832_xxAA



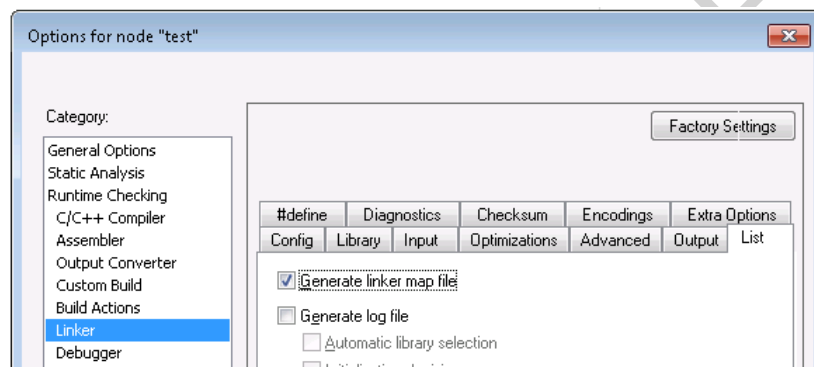
Create an ICF (ILINK Configuration File) file and add it into the Config tab of the Linker Page. ICF file is used to define the address on which the binary file will be programmed.



Edit the ICF file and add following line in the file.

```
place at address mem:0x0200000 { readonly section psp_image_section };
```

Step 3: Go to the List tab. Generate a linker map file here in order to check whether the binary file has been located at the desired address.



Detail of the implementation is described in following reference.

<https://www.iar.com/support/tech-notes/general/programming-binary-files-into-flash-within-ewarm/>

<https://www.iar.com/support/tech-notes/linker/using-a-const-char-pointer-to-access-a-binary-image/>

<https://www.iar.com/contentassets/554f673197f74bfa9059dd4094d61bee/instructions-how-to-flash-one-bin-file-with-ewarm.pdf>

Example of ICF file is

```
/*###ICF### Section handled by ICF editor, don't touch! ****/
/*-Editor annotation file-*/
/* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_v1_0.xml" */
/*-Specials-*/
define symbol __ICFEDIT_intvec_start__ = 0x00000000;
/*-Memory Regions-*/
define symbol __ICFEDIT_region_ROM_start__ = 0x00000000;
define symbol __ICFEDIT_region_ROM_end__ = 0x0007FFFF;
```

```

define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
define symbol __ICFEDIT_region_RAM_end__   = 0x2000FFFF;
/*-Sizes-*/
define symbol __ICFEDIT_size_cstack__ = 0x400;
define symbol __ICFEDIT_size_heap__   = 0x200;
/**** End of ICF editor section. ###ICF###*/

define memory mem with size = 4G;
define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];

define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };

initialise by copy { readwrite };
//initialise by copy with packing = none { section __DLIB_PERTHREAD }; // Required in a multi-threaded application
do not initialise { section .noinit };

place at address mem: __ICFEDIT_intvec_start__ { readonly section .intvec };
place at address mem: 0x020000 { readonly section psp_image_section };

place in ROM_region { readonly };
place in RAM_region { readwrite, block CSTACK, block HEAP };

```

3.6.2 Including header and helper files

It is recommended to put the header and helper files in the same directory, so the files can be properly built in compiling process. To make the library functions available to the project, the header file “psp.h” should be included in application project by specifying

```
#include "psp.h"
```

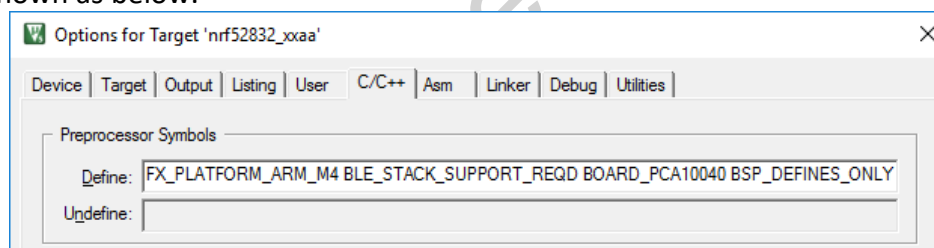
Putting following files in the appropriate directories.

```
psp.h
psp_map.h
psp.c
fx_datatypes.h
psp.bin
psp.hex
```

For the library running on ARM Cortex_M4F, it should define a macro for the platform with

```
#define FX_PLATFORM_ARM_M4
```

In Keil, you can achieve the same task by adding FX_PLATFORM_ARM_M4 in C/C++ tag of Options for target. It is shown as below.



A Appendix: Typical Integration Issues

A.1 Linking Library in Keil

Sometimes the library, when linking using Keil compiler, the following linking error will appear:

```
.\_build\nrf52832_xxaa.axf: Error: L6242E: Cannot link object fx_graph.o as its attributes are incompatible with the image attributes.  
... wchart-16 clashes with wchart-32.
```

This is a known limitation of combining libraries compiled with different compilers. The message itself can be safely ignored without consequence. To suppress this error in Keil, set “--diag_suppress L6242” in the Misc controls dialog box under the Options for Target ‘xxx’ Linker tab.

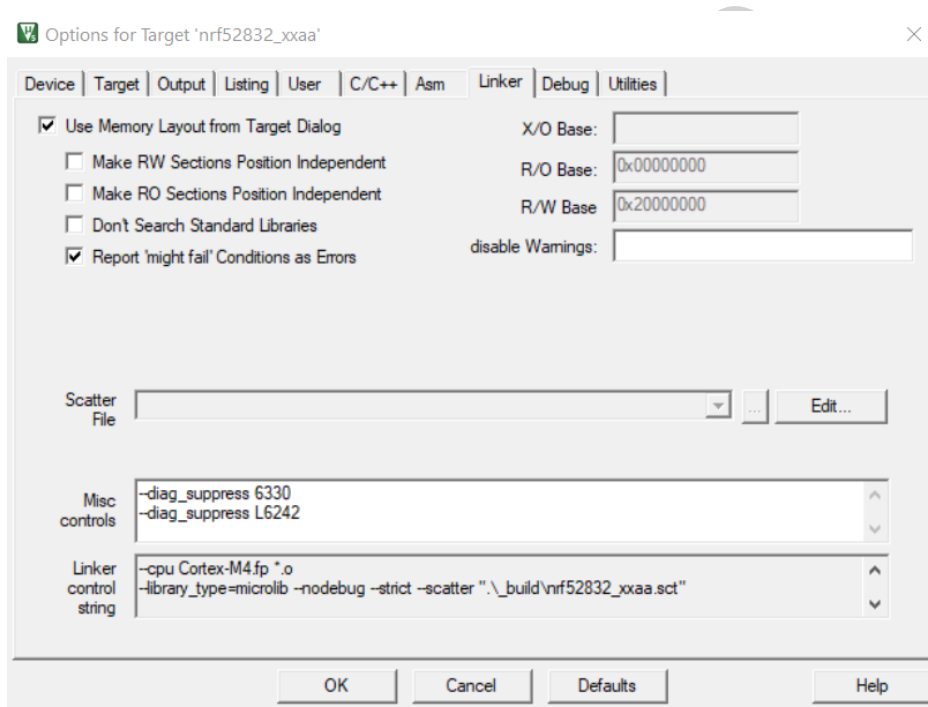


Fig 8. Suppress wchar error in Keil

Legal information

Disclaimers

This document may be subject to change without notice. Philips Electronics Nederland B.V. does not give any representations or warranties as to the accuracy or completeness of information included herein and shall have no liability for the consequences of use of such information.

This document is provided “as is” with no warranties, expressed or implied, including but not limited to any implied warranty of merchantability, fitness for a particular purpose, or freedom from infringement. Koninklijke Philips N.V. or its Affiliates may have patents or pending patent applications, or other intellectual property rights that relate to the described subject matter. Philips and its logo are trademarks of Koninklijke Philips N.V. All other names are the trademark or registered trademarks of their respective holders. The furnishing of this document does not provide any license, expressed or implied, by estoppels or otherwise, to any such patents, trademarks or copyrights, or other intellectual property rights. Philips Electronics Nederland B.V. assumes no responsibility for errors or omissions in this document; nor does Philips Electronics Nederland B.V. make any

commitment to update the information contained herein. This document is subject to change without notice.

Patents

Notice is herewith given that the subject device uses one or more patents and that each of these patents may have corresponding patents in other jurisdictions.

Philips has been awarded a number of patents on the technology used in the Philips Biosensing Platform offering in Europe, Asia and the United States.

Trademarks

Notice: All referenced brands, service names and trademarks are property of their respective owners.

CoreKinect LLC